جامعة أم القرى

UMM AL-QURA UNIVERSITY

Toy and Surprise Store

Student names & ID

Lujain Bakkar: 445005807
Reema Khalid :446000177
Meral Alansari:446001757

Course name

Introduction to Database

Instructor's name

Arwa Halawani

College of Computing

Department of Computer Science and Artificial Intelligence

Table of Contents:

1.(Cover Page) 1

2.(Table of Contents) 3

3. (Assigned Responsibilities) 4

Section 1: Conceptual and Logical Design

4. (Scenario Description) 5

5. ER Diagram 6

6.Relational Schema (ER) 7

Section 2: Implementation

7. (Creating the Database& Creating Tables & Applying

Constraints: Data Types, PK, FK, UNIQUE, NOT NULL, CHECK, UPDATE, DELETE)
..... 8

8. DESCRIBE FOR ALL TABLES……...13

9. SQL (SQL Queries) 14

Assigned Responsibilities

Lujain Bakkar	Meral Alansari	Reema Khalid
ER Diagram and Converting to Relational Schema	Writing the scenario	MySQL CODE

"Every team member contributed to all stages of the project, from writing the scenario to implementing the code, designing the ER Diagram, and converting it into the Relational Schema. The work was entirely collaborative, and no one completed any task entirely on their own. Therefore, all members share joint responsibility for the project."

Conceptual and Logical Design

Database Description

Toy and Surprise is a specialty store dedicated to high-quality toys, focusing on providing trendy toys from global manufacturers. The database is designed to organize information related to customers, orders, products, and employees, while defining the relationships among these entities to ensure efficient and accurate business operations.

Entity: Customers

This entity stores data about customers who interact with the store, either through the store or online platforms.

It includes:

•Customer ID (Primary key)

•Name (First Name, Middle Name (Optional), Last Name (Optional)) (composite)

•Email(unique)

•Total Points

•Membership Level (Derived)

Membership Level: used to determine each customer's loyalty level and to provide discounts based on visit frequency and total purchase value.

Entity: Orders

This entity contains all details related to purchase transactions made by customers.

It includes:

•Order ID (Primary Key)

•Order Date

•Status (Processing, Confirmed, Delivered, Pending, Cancelled, Failed) (Multi-valued)

•Payment Method (Cash, Card) (Composite)

•Total Amount

Entity: Products

This entity stores information about the toys and related products sold by the store.

It includes:

•Product ID (Primary Key)

•Product Name

•Description

•Price

•Available Quantity

•Age Group (0-6 months, 6-12 months, +3 years, 3-6 years, 6-12 years, +12 years) (Composite)

•Gender Target (Male, Female) (Composite)

Entity: Employees

This entity contains information about all employees working in the store.

It includes:

•Employee ID (Primary Key)

•Name (First Name, Middle Name, Last Name) (composite)

•Email (Unique)

•Phone Number (Multi-Valued)

•Salary

•Hiring Date

•Job Role (Composite)

Job Role: include Call Center, Cashier, Salesperson, Marketer, Team Supervisor, Event Coordinator, Exhibition Manager.

Relationships:

Include:

Each order can include several products, and each product can be included in several orders (Many (Optional)-to-Many (Mandatory)).

Try:

A product can be tried by many customers, and each customer can try any product (Many (Optional)-to-Many (Optional)).

Handle:

An employee can handle multiple orders, and each order can be handled by multiple employees (Many (Mandatory)-to-Many (Mandatory)).

Assist:

A customer can be assisted by multiple employees, and each employee can assist several customers (Many (Mandatory)-to-Many (Mandatory)).

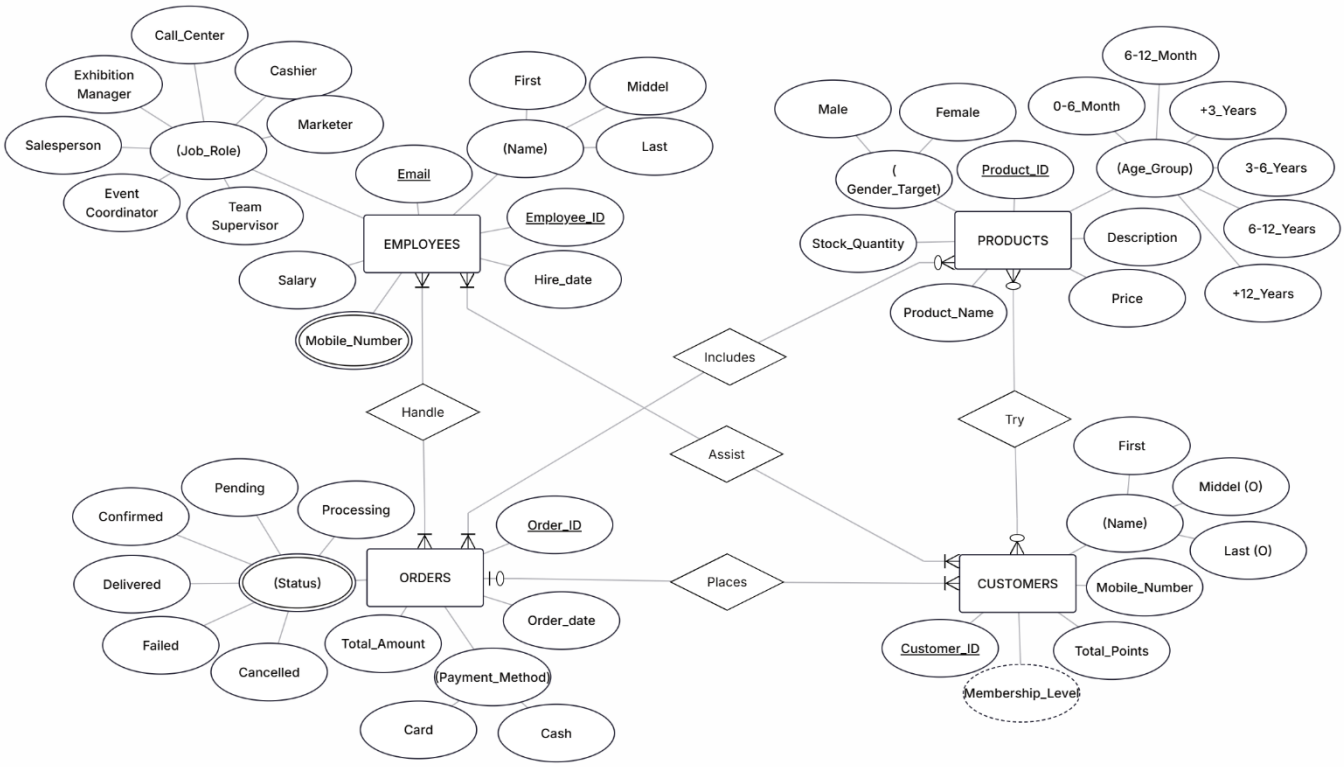
Place:

A customer can place many orders, while each order belongs to one customer (One (Mandatory)-to-Many (Optional)).

Conceptual and Logical Design

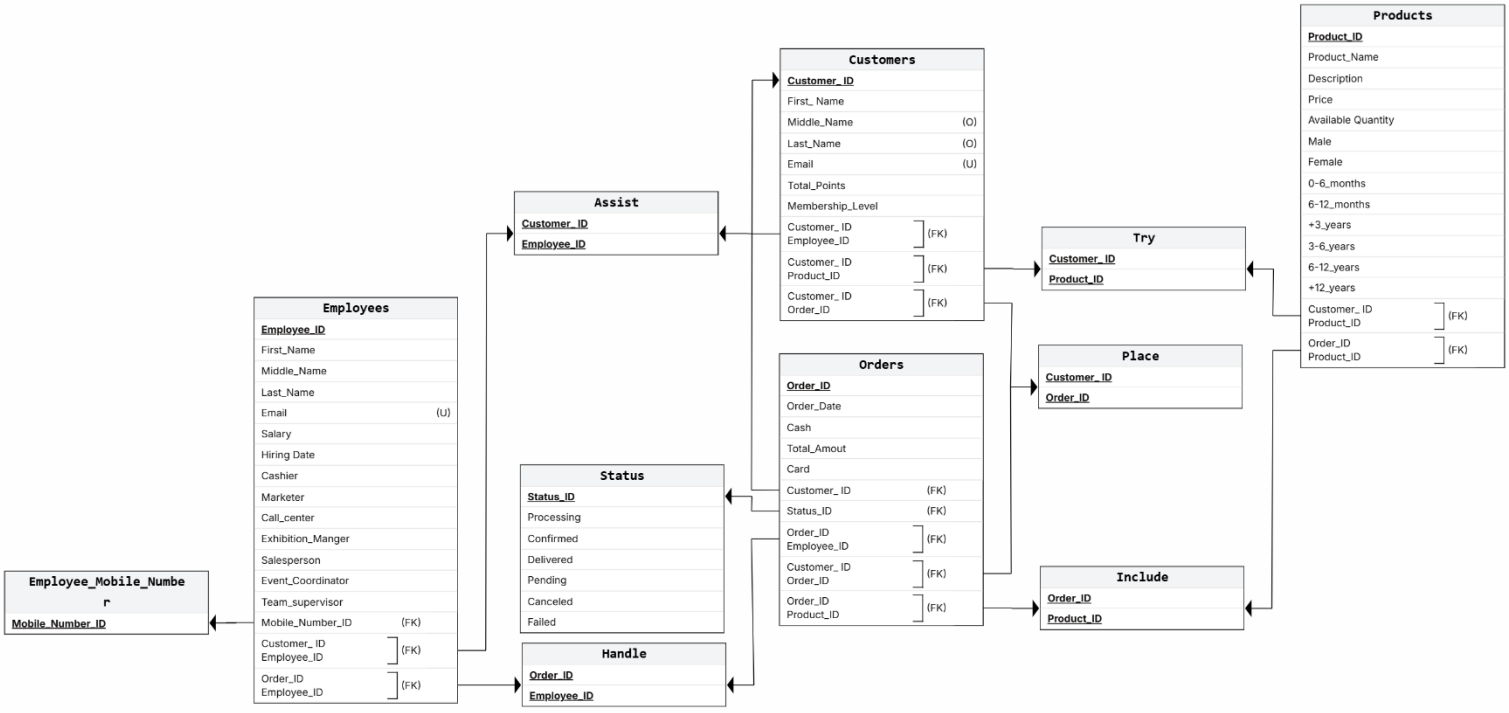
ER Diagram:

The Entity–Relationship Diagram (ERD) for the Toy and Surprise Store consists of 4 main entities, connected through 5 relationships that include one-to-many and many-to-many associations. The diagram contains more than 30 attributes in total, distributed across the entities. Among these attributes, there are several composite attributes, 2 multi-valued attributes, and 1 derived attribute. This ERD provides a clear conceptual view of how data is structured and how entities interact within the system.



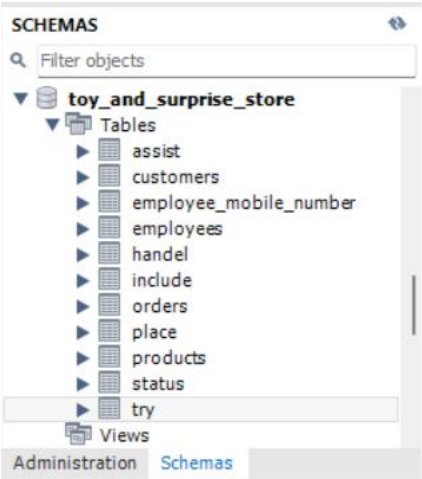
Relational Schema:

The database schema is the logical representation of the ERD and was designed using a relational modeling tool. It includes 4 core tables derived from the main entities and multiple associative tables to resolve the many-to-many relationships. The schema defines primary keys for each table and several foreign keys to enforce referential integrity. In addition, separate tables are used to handle multi-valued and many-to-many attributes. ensuring normalization and reducing data redundancy while supporting efficient database operations.



Creating the Database & Creating Tables& Applying Constraints

Creating the Database & Creating Tables



Create Customers Table

```
CREATE TABLE Customers (
    Customer_ID  VARCHAR(10) PRIMARY KEY,
    First_Name VARCHAR(25),
    Middle_Name VARCHAR(25),
    Last_Name VARCHAR(25),
    Email VARCHAR(80) UNIQUE NOT NULL,
    Total_Points INT DEFAULT 0 CHECK (Total_Points >= 0),
    Membership_Level char(30)
);
```

Create Employees Table

```
CREATE TABLE Employees (
    Employee_ID varchar(10) PRIMARY KEY,
    First_Name VARCHAR(25) NOT NULL,
    Middle_Name VARCHAR(25) NOT NULL,
    Last_Name VARCHAR(25) NOT NULL,
    Email VARCHAR(80) UNIQUE NOT NULL,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0),
    Hiring_Date DATE NOT NULL,
    Job_Role ENUM('Marketer','Cashier','Call Center','Exhibition Manager','Salesperson','Event Coordinator','Team Supervisor') NOT NULL
```

Create Products Table

```sql
CREATE TABLE Products (
    Product_ID  VARCHAR(10) PRIMARY KEY,
    Product_Name VARCHAR(30) NOT NULL,
    Description TEXT,
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    Available_Quantity INT DEFAULT 0 CHECK (Available_Quantity >= 0),
    Target_Gender ENUM ('Male','Female') NOT NULL,
    Target_Age ENUM('0-6 months', '6-12 months', '1-3 years', '3-6 years', '6-12 years', '+12 years') NOT NULL
);
```

Update Products Table

```sql
-- Update the price of a specific product by increasing it by 10
UPDATE Products
SET Price = Price + 10
WHERE Product_ID = 'P001';
```

Create Employee Mobile Number Table

```sql
CREATE TABLE Employee_Mobile_Number (
    Mobile_Number_ID VARCHAR(15),
     Employee_ID varchar(10),
    PRIMARY KEY (Mobile_Number_ID, Employee_ID),
    FOREIGN KEY (Employee_ID) REFERENCES Employees(Employee_ID)
    ON DELETE RESTRICT
);
```

ON DELETE RESTRICT means: "we cannot delete a record in the parent table if there are related records in the child table." For example, if a customer has orders, you can't delete the customer until all their orders are deleted. In your case, the customer was deleted successfully because they had no orders, so the restriction didn't block the deletion.
ON DELETE CASCADE means: "If you delete a record in the parent table, all related records in the child table are automatically deleted." So, if we delete a customer, all their orders disappear too, without needing to delete them manually.

Create Status Table

```sql
CREATE TABLE Status (
    Status_ID VARCHAR(10) PRIMARY KEY,
     Status_Name ENUM('Processing','Confirmed','Delivered','Pending','Canceled','Failed') NOT NULL

);
```

Create Orders Table

```sql
CREATE TABLE Orders (
    Order_ID VARCHAR(10) PRIMARY KEY,
    Order_Date DATE NOT NULL,
    Payment_Method ENUM('Cash', 'Card') NOT NULL,
    Total_Amount DECIMAL(10,2) NOT NULL CHECK (Total_Amount >= 0),
    Customer_ID VARCHAR(10),
    Status_ID VARCHAR(10),
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Status_ID) REFERENCES Status(Status_ID)
    ON DELETE RESTRICT
);
```

Crate Assist Table

```sql
CREATE TABLE Assist (
    Customer_ID VARCHAR(10),
    Employee_ID VARCHAR(10),
    PRIMARY KEY (Customer_ID, Employee_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Employee_ID) REFERENCES Employees(Employee_ID)
    ON DELETE RESTRICT
);
```

Crate Handle Table

```sql
CREATE TABLE Handel (
    Employee_ID VARCHAR(10),
    Order_ID VARCHAR(10),
    PRIMARY KEY (Employee_ID, Order_ID),
    FOREIGN KEY (Employee_ID) REFERENCES Employees(Employee_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    ON DELETE RESTRICT
);
```

Crate Include Table

```sql
CREATE TABLE Include (
    Order_ID VARCHAR(10),
    Product_ID VARCHAR(10),
    PRIMARY KEY (Order_ID, Product_ID),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
    ON DELETE RESTRICT
);
```

Create Try Table

```sql
CREATE TABLE Try (
    Customer_ID VARCHAR(10),
    Product_ID VARCHAR(10),
    PRIMARY KEY (Customer_ID, Product_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
    ON DELETE RESTRICT
);
```

Delete Specific Record from Try Table

```sql
-- Remove a specific product trial record for a customer
DELETE FROM Try
WHERE Customer_ID = 'C026'
  AND Product_ID = 'P005';
```

Create Place Table

```sql
CREATE TABLE Place (
    Customer_ID VARCHAR(10),
    Order_ID VARCHAR(10),
    PRIMARY KEY (Customer_ID, Order_ID),
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
    ON DELETE RESTRICT,
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    ON DELETE RESTRICT
);
```

DESCRIBE FOR ALL TABLES

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

Field	Type	Null	Key	Default	Extra
Customer_ID	varchar(10)	NO	PRI	NULL	
First_Name	varchar(25)	NO		NULL	
Middle_Name	varchar(25)	YES		NULL	
Last_Name	varchar(25)	YES		NULL	
Email	varchar(80)	NO	UNI	NULL	
Total_Points	int	YES		0	
Membership_Level	char(30)	YES		NULL	

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Employee_ID	varchar(10)	NO	PRI	NULL	
First_Name	varchar(25)	NO		NULL	
Middle_Name	varchar(25)	NO		NULL	
Last_Name	varchar(25)	NO		NULL	
Email	varchar(80)	NO	UNI	NULL	
Salary	decimal(10,2)	NO		NULL	
Hiring_Date	date	NO		NULL	
Job_Role	enum('Marketer','Cashier','Call Center','Exhibitio...	NO		NULL	

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap C

Field	Type	Null	Key	Default	Extra
Mobile_Number_ID	varchar(15)	NO	PRI	NULL	
Employee_ID	varchar(10)	NO	PRI	NULL	

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Product_ID	varchar(10)	NO	PRI	NULL	
Product_Name	varchar(30)	NO		NULL	
Description	text	YES		NULL	
Price	decimal(10,2)	NO		NULL	
Available_Quantity	int	YES		0	
Target_Gender	enum('Male','Female')	NO		NULL	
Target_Age	enum('0-6 months','6-12 months','1-3 years','3-...	NO		NULL	

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
Status_ID	varchar(10)	NO	PRI	NULL	
Status_Name	enum('Processing','Confirmed','Delivered','Pendi...	NO		NULL	

```
DESCRIBE Customers;
DESCRIBE Employees;
DESCRIBE Employee_Mobile_Number;
DESCRIBE Products;
DESCRIBE Status;
DESCRIBE Orders;
DESCRIBE Assist;
DESCRIBE Handel;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Field	Type	Null	Key	Default	Extra
Order_ID	varchar(10)	NO	PRI	NULL	
Order_Date	date	NO		NULL	
Payment_Method	enum('Cash','Card')	NO		NULL	
Total_Amount	decimal(10,2)	NO		NULL	
Customer_ID	varchar(10)	YES	MUL	NULL	
Status_ID	varchar(10)	YES	MUL	NULL	

```
249 • DESCRIBE Products;
250 • DESCRIBE Status;
251 • DESCRIBE Orders;
252 • DESCRIBE Assist;
253 • DESCRIBE Handel;
254 • DESCRIBE Include;
255 • DESCRIBE Try;
256 • DESCRIBE Place;
257
```

Result Grid | Filter Rows: | Export: | Wra

Field	Type	Null	Key	Default	Extra
Customer_ID	varchar(10)	NO	PRI	NULL	
Employee_ID	varchar(10)	NO	PRI	NULL	

```
249 • DESCRIBE Products;
250 • DESCRIBE Status;
251 • DESCRIBE Orders;
252 • DESCRIBE Assist;
253 • DESCRIBE Handel;
254 • DESCRIBE Include;
255 • DESCRIBE Try;
256 • DESCRIBE Place;
257
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Employee_ID	varchar(10)	NO	PRI	NULL	
Order_ID	varchar(10)	NO	PRI	NULL	

```
249 • DESCRIBE Products;
250 • DESCRIBE Status;
251 • DESCRIBE Orders;
252 • DESCRIBE Assist;
253 • DESCRIBE Handel;
254 • DESCRIBE Include;
255 • DESCRIBE Try;
256 • DESCRIBE Place;
257
```

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Extra
Order_ID	varchar(10)	NO	PRI	NULL	
Product_ID	varchar(10)	NO	PRI	NULL	

```
249 • DESCRIBE Products;
250 • DESCRIBE Status;
251 • DESCRIBE Orders;
252 • DESCRIBE Assist;
253 • DESCRIBE Handel;
254 • DESCRIBE Include;
255 • DESCRIBE Try;
256 • DESCRIBE Place;
257
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Customer_ID	varchar(10)	NO	PRI	NULL	
Product_ID	varchar(10)	NO	PRI	NULL	

```
249 • DESCRIBE Products;
250 • DESCRIBE Status;
251 • DESCRIBE Orders;
252 • DESCRIBE Assist;
253 • DESCRIBE Handel;
254 • DESCRIBE Include;
255 • DESCRIBE Try;
256 • DESCRIBE Place;
257
```

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
Customer_ID	varchar(10)	NO	PRI	NULL	
Order_ID	varchar(10)	NO	PRI	NULL	

SQL (Queries)

Select Queries:

First One Reema:

```sql
-- This query displays customers who have more than 1000 loyalty points
SELECT Customer_ID, First_Name, Last_Name, Total_Points
FROM Customers
WHERE Total_Points > 1000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

Customer_ID	First_Name	Last_Name	Total_Points
C024	Sara	Al-Khalid	1200
C028	Fahd	Al-Qarni	1800
C030	Abdulaziz	Al-Fahid	1100
C032	Turki	Al-Shehri	1250
NULL	NULL	NULL	NULL

Second One Reema:

```sql
-- This query displays products that cost less than 100
SELECT Product_ID, Product_Name, Price
FROM Products
WHERE Price < 100;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

Product_ID	Product_Name	Price
P002	Talking Moving Doll	89.50
P003	Building Blocks Game	75.00
P005	Spiderman Toys	55.00
P008	Intelligence Puzzle	45.50
P010	Water Gun Set	39.99
NULL	NULL	NULL

Select Queries:

First One Lujain:

```
-- Retrieves Platinum member to identify high-value customers for exclusive offers.
SELECT Customer_ID, Email, Total_Points, Membership_Level
FROM Customers
WHERE Membership_Level = 'Platinum';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Customer_ID	Email	Total_Points	Membership_Level
C024	C024@gmail.com	1200	Platinum
C028	C028@gmail.com	1800	Platinum
C030	C030@gmail.com	1100	Platinum
C032	C032@gmail.com	1250	Platinum
NULL	NULL	NULL	NULL

Second One Lujain:

```
-- Finds products that out-of-stock to manage stock replenishment.
SELECT Product_ID, Product_Name, Price, Available_Quantity
FROM Products
WHERE Available_Quantity = 0;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Product_ID	Product_Name	Price	Available_Quantity
NULL	NULL	NULL	NULL

Select Queries:

First One Meral:

```sql
-- to show product with price higher than 100
SELECT Product_Name, Price
FROM products
WHERE price > 100;
```

Result Grid | Filter Rows: | Export: | Wrap

Order_ID	Order_Date	First_Name	Last_Name
O01	2024-10-01	Ahmed	Al-Zaid
O02	2024-10-02	Sara	Al-Khalid
O03	2024-10-03	Mohammed	Al-Saad
O04	2024-10-04	Lina	Al-Jabri
O05	2024-10-05	Nawaf	Al-Ghanim
O06	2024-10-06	Fahd	Al-Qarni
O07	2024-10-07	Hana	Al-Otaibi
O08	2024-10-08	Abdulaziz	Al-Fahid
O09	2024-10-09	Mai	Al-Manea
O10	2024-10-10	Turki	Al-Shehri

Second One Meral:

```sql
-- Identifies employees hired in 2024 for administrative review.
SELECT First_Name, Last_Name, Hiring_Date
FROM Employees
WHERE Hiring_Date >= '2024-01-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name	Hiring_Date
Omar	Al-Ghamdi	2024-03-01
Reem	Al-Amri	2024-01-10

Aggregate Functions Queries

First One Reema:

```sql
-- COUNT() with GROUP BY
-- Write SQL statement
-- This query counts how many orders were made for each payment method
SELECT Payment_Method, COUNT(Order_ID) AS total_orders
FROM Orders
GROUP BY Payment_Method;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Payment_Method	total_orders
Card	5
Cash	5

Second One Reema:

```sql
-- AVG() aggregate function
-- This query calculates the average salary of all employees
SELECT AVG(Salary) AS average_salary
FROM Employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

average_salary
8290.000000

Aggregate Functions Queries

First One Lujain:

```sql
-- COUNT() aggregate function
-- with GROUP BY
-- Counts how many employees work in each job type.
SELECT Job_Role,
       COUNT(*) AS Employee_Count
FROM Employees
GROUP BY Job_Role;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

Job_Role	Employee_Count
Salesperson	3
Team Supervisor	3
Cashier	2
Call Center	2

Second One Lujain:

```sql
-- COUNT() aggregate function
-- Counts how many customers placed orders on 2024-10-01.
SELECT COUNT(DISTINCT Customer_ID) AS Customer_Count
FROM Orders
WHERE Order_Date = '2024-10-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_Count
1

Aggregate Functions Queries

First One Meral:

```sql
-- COUNT() aggregate function
-- With GROUP BY
-- Counts how many customers are in each membership level to analyze loyalty.
SELECT Membership_Level, COUNT(*) AS Total_Customers
FROM Customers
GROUP BY Membership_Level;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Membership_Level	Total_Customers
Gold	4
Platinum	4
Silver	2

Second One Meral:

```sql
-- COUNT(), AVG() aggregate functions
-- Calculates total store revenue and the average order value for financial reports.
SELECT SUM(Total_Amount) AS Total_Revenue,
       AVG(Total_Amount) AS Average_Order_Value
FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Revenue	Average_Order_Value
1954.97	195.497000

Joint Queries

First One Reema:

```sql
-- INNER JOIN between Orders and Customers
-- This query displays orders with the customer full name
SELECT O.Order_ID, O.Order_Date, C.First_Name, C.Last_Name
FROM Orders O
INNER JOIN Customers C
ON O.Customer_ID = C.Customer_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Order_Date	First_Name	Last_Name
O01	2024-10-01	Ahmed	Al-Zaid
O02	2024-10-02	Sara	Al-Khalid
O03	2024-10-03	Mohammed	Al-Saad
O04	2024-10-04	Lina	Al-Jabri
O05	2024-10-05	Nawaf	Al-Ghanim
O06	2024-10-06	Fahd	Al-Qarni
O07	2024-10-07	Hana	Al-Otaibi
O08	2024-10-08	Abdulaziz	Al-Fahid
O09	2024-10-09	Mai	Al-Manea
O10	2024-10-10	Turki	Al-Shehri

Second One Reema:

```sql
-- INNER JOIN between Include and Products
-- This query displays each order with its products name
SELECT I.Order_ID, P.Product_Name
FROM Include I
INNER JOIN Products P
ON I.Product_ID = P.Product_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Product_Name
O01	Remote Control Car
O02	Remote Control Car
O04	Remote Control Car
O07	Talking Moving Doll
O02	Building Blocks Game
O03	Building Blocks Game
O04	Electric Train
O07	Spiderman Toys
O06	Drone Aircraft
O05	Kids Cooking Game
O02	Intelligence Puzzle

Joint Queries

First One Lujain:

```sql
-- JOIN between Try and Products
-- Shows product trials with actual product names.
SELECT T.Customer_ID, P.Product_Name
FROM Try T
JOIN Products P ON T.Product_ID = P.Product_ID
ORDER BY T.Customer_ID, P.Product_Name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

Customer_ID	Product_Name
C023	Building Blocks Game
C023	Remote Control Car
C024	Talking Moving Doll
C025	Electric Train
C026	Spiderman Toys
C027	Drone Aircraft
C028	Kids Cooking Game
C029	Intelligence Puzzle
C030	Robot Dog
C031	Water Gun Set

Second One Lujain:

```sql
-- JOIN between Customers and Orders
-- Shows each customer's total orders.
SELECT C.Customer_ID, C.First_Name,
       COUNT(O.Order_ID) AS Order_Count
FROM Customers C
JOIN Orders O ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID, C.First_Name;
```

Result Grid | Filter Rows: | Export: | Wrap

Customer_ID	First_Name	Order_Count
C023	Ahmed	1
C024	Sara	1
C025	Mohammed	1
C026	Lina	1
C027	Nawaf	1
C028	Fahd	1
C029	Hana	1
C030	Abdulaziz	1
C031	Mai	1
C032	Turki	1

Joint Queries

First One Meral:

```sql
-- join between customer and order
-- Shows customer names along with their order details to track purchases.
SELECT C.First_Name, O.Order_Date, O.Total_Amount
FROM Customers C
JOIN Orders O ON C.Customer_ID = O.Customer_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Order_Date	Total_Amount
Ahmed	2024-10-01	149.99
Sara	2024-10-02	245.00
Mohammed	2024-10-03	75.00
Lina	2024-10-04	374.99
Nawaf	2024-10-05	120.00
Fahd	2024-10-06	299.99
Hana	2024-10-07	165.00
Abdulaziz	2024-10-08	89.50
Mai	2024-10-09	240.00
Turki	2024-10-10	195.50

Second One Meral:

```sql
--  join between handle and employee
-- SHOW THE EMPLOYEE WHO HANDLED THE ORDERS
SELECT E.First_Name, H.Order_ID
FROM Employees E
JOIN Handel H ON E.Employee_ID = H.Employee_ID;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

First_Name	Order_ID
Khalid	O01
Khalid	O02
Nora	O02
Omar	O03
Lama	O04
Faisal	O05
Maha	O06
Abdullah	O07
Reem	O08
Majed	O09
Nouf	O10

Subquery Queries

First One Reema:

```sql
-- Subquery using AVG()
-- This query displays products that are more expensive than the average product price
SELECT Product_Name, Price
FROM Products
WHERE Price > (
    SELECT AVG(Price)
    FROM Products
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Product_Name	Price
Remote Control Car	149.99
Electric Train	199.99
Drone Aircraft	299.99
Robot Dog	179.99

Second One Reema:

```sql
-- Subquery using IN
-- This query displays customers who have placed at least one order
SELECT Customer_ID, First_Name, Last_Name
FROM Customers
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Orders
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Customer_ID	First_Name	Last_Name
C023	Ahmed	Al-Zaid
C024	Sara	Al-Khalid
C025	Mohammed	Al-Saad
C026	Lina	Al-Jabri
C027	Nawaf	Al-Ghanim
C028	Fahd	Al-Qarni
C029	Hana	Al-Otaibi
C030	Abdulaziz	Al-Fahid
C031	Mai	Al-Manea
C032	Turki	Al-Shehri
NULL	NULL	NULL

Subquery Queries

First One Lujain:

```sql
-- Subquery using IN
-- Shows products priced below average product price.
SELECT Product_Name, Price
FROM Products
WHERE Price IN (
    SELECT Price
    FROM Products
    WHERE Price < (
        SELECT AVG(Price)
        FROM Products)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

Product_Name	Price
Talking Moving Doll	89.50
Building Blocks Game	75.00
Spiderman Toys	55.00
Kids Cooking Game	120.00
Intelligence Puzzle	45.50
Water Gun Set	39.99

Second One Lujain:

```sql
-- Subquery using MAX
-- Shows details of the order with the highest total amount.
SELECT Order_ID, Customer_ID, Total_Amount
FROM Orders
WHERE Total_Amount IN (
    SELECT MAX(Total_Amount)
    FROM Orders
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

Order_ID	Customer_ID	Total_Amount
O04	C026	374.99
NULL	NULL	NULL

Subquery Queries

First One Meral:

```sql
-- Subquery using IN
-- get the name of the customers who got helped bt employee number E027
SELECT First_Name, Last_Name
FROM Customers
WHERE Customer_ID IN (SELECT Customer_ID FROM Assist WHERE Employee_ID = 'E027');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name
Nawaf	Al-Ghanim

Second One Meral:

```sql
-- Subquery using NOT IN
-- Lists products that have never been included in any order.
SELECT Product_Name
FROM Products
WHERE Product_ID NOT IN (SELECT DISTINCT Product_ID FROM Include);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Product_Name
Robot Dog
Water Gun Set

View Queries

Reema:

```sql
-- This view displays delivered orders with customer information
CREATE VIEW Delivered_Orders AS
SELECT O.Order_ID, O.Order_Date, O.Total_Amount,
       C.First_Name, C.Last_Name
FROM Orders O
INNER JOIN Customers C
ON O.Customer_ID = C.Customer_ID
WHERE O.Status_ID = 'S03';

-- Display data from the view
SELECT * FROM Delivered_Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Order_Date	Total_Amount	First_Name	Last_Name
O01	2024-10-01	149.99	Ahmed	Al-Zaid

Lujain:

```sql
-- helping identify what Product is Below the Average.
CREATE VIEW Below_Average_Price_Products AS
SELECT Product_Name, Price
FROM Products
WHERE Price < (
    SELECT AVG(Price)
    FROM Products
);

-- Display data from the view
SELECT * FROM Below_Average_Price_Products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Product_Name	Price
Talking Moving Doll	89.50
Building Blocks Game	75.00
Spiderman Toys	55.00
Kids Cooking Game	120.00
Intelligence Puzzle	45.50
Water Gun Set	39.99

Meral:

```sql
-- Reates a summary report for each customer's total spending and order count.
CREATE VIEW Customer_Activity_Report AS
SELECT C.First_Name, C.Last_Name, COUNT(O.Order_ID) AS Orders_Count,
 SUM(O.Total_Amount) AS Total_Spent
FROM Customers C
LEFT JOIN Orders O ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID, C.First_Name, C.Last_Name;

-- Display data from the view
SELECT * FROM Customer_Activity_Report;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name	Orders_Count	Total_Spent
Ahmed	Al-Zaid	1	149.99
Sara	Al-Khalid	1	245.00
Mohammed	Al-Saad	1	75.00
Lina	Al-Jabri	1	374.99
Nawaf	Al-Ghanim	1	120.00
Fahd	Al-Qarni	1	299.99
Hana	Al-Otaibi	1	165.00

جامعة أم القرى

UMM AL-QURA UNIVERSITY